PET BREED ML PIPELINE

Theory Guide: Training and Evaluation

Oxford-IIIT Pet · PyTorch · ResNet-18 · Transfer Learning

A practical theory guide to the repository's end-to-end machine-learning pipeline for classifying cat and dog breeds.

Focus: Oxford-IIIT Pet dataset, image preprocessing, ResNet-18, transfer learning, training, validation, test evaluation, metrics, checkpoints, and prediction.

This guide covers the Stage 1 training pipeline only. The separate pet-model-deployment repository contains the later fine-tuning and serving stage.

Repository: https://github.com/ApexHyper2007/pet-breed-ml-pipeline

Use this guide to understand the design decisions, explain the code during a viva, and debug common model-training problems.

Project overview and scope

The project takes a labeled image dataset and learns a mapping from an image to one of 37 cat or dog breed classes. It is an image-classification problem: the model returns class scores, and the highest-scoring class becomes the top prediction.

Pipeline stages

- Dataset acquisition and integrity checks.
- Train/validation/test split manifests.
- Image decoding, resizing, augmentation, tensor conversion, and normalization.
- A ResNet-18 backbone initialized from pretrained ImageNet weights.
- A replacement classifier head with 37 outputs.
- Training on the training split and checkpoint selection using validation accuracy.
- Evaluation on a held-out test set.
- Single-image inference through a prediction script.

The central engineering idea is separation of concerns: dataset code handles examples and labels; model code defines the neural network; training code optimizes parameters; evaluation code measures generalization; prediction code applies the saved model to new images.

Problem formulation: fine-grained classification

Let x represent an RGB image and y its correct breed label. The label belongs to one of 37 categories. A neural network $f_\theta(x)$ produces 37 logits, where θ represents the learned model parameters.

```
image x → preprocessing → ResNet-18 → 37 logits
          → softmax scores → ranked breed labels
```

A logit is an unnormalized score. Softmax maps the vector of logits to non-negative values that sum to one. The class with the largest logit also has the largest softmax score, so argmax can select the top class.

Closed-set assumption

The classifier assumes the correct answer is one of the 37 known breeds. It will still rank known classes for an unknown breed, mixed breed, blurred photo, or unrelated image. The system does not automatically detect out-of-distribution inputs.

Why it is fine-grained

Some breeds have similar fur patterns, face shapes, or body proportions. This makes the problem more challenging than simply classifying an image as cat versus dog.

Oxford-IIIT Pet dataset

The Oxford-IIIT Pet dataset contains images of cats and dogs across 37 breed categories. The repository's documented workflow uses 7,349 images and label annotations. Dataset quality directly affects the model: mislabeled images, missing files, duplicates, or inconsistent class indices can distort both training and reported results.

Labels and class indices

A dataset label is often stored as an integer, while humans recognize a breed name. A class-to-index mapping links those two representations. Training, evaluation, and inference must all use the same mapping.

Data integrity checks

- Confirm every manifest path points to an existing image.
- Check images can be decoded and converted to RGB.
- Check labels fall within the expected 37-class range.
- Check for duplicate images across data splits where possible.
- Confirm that class names and numerical indices remain stable.

A model can produce numerically valid predictions while the displayed breed name is wrong if the mapping changes between training and inference.

Train, validation, and test sets

Training examples are used to calculate gradients and update weights. Validation examples are used to compare checkpoints and tune hyperparameters. Test examples are held back for a final estimate of generalization.

Why three splits?

If the test set is repeatedly consulted during model selection, information from it indirectly influences development. This causes test-set leakage and can make the final result look better than true performance.

Stratification and fixed manifests

A stratified split tries to preserve class proportions in each subset. Saving split manifests makes the experiment repeatable and ensures later comparisons use the same examples.

Reproducibility checklist

- Record the random seed and actual file lists.
- Record preprocessing and augmentation settings.
- Record model architecture, pretrained weights, optimizer, and learning rate.
- Record the checkpoint used for final test evaluation.

If images from the same source or near-duplicate photos cross splits, the evaluation can be overly optimistic. Split strategy should consider the structure of the dataset, not only the count of images.

Image decoding and preprocessing

JPEG and other image formats must be decoded into pixel arrays before a neural network can process them. The pipeline opens each file, converts it to RGB, applies transforms, and returns a tensor paired with its class label.

RGB representation

RGB images have three channels: red, green, and blue. Converting images to RGB ensures a consistent channel count even if the original file is grayscale or has a different color mode.

Resizing and crops

ResNet-18 is commonly trained with 224 × 224 inputs. Resizing and cropping standardize the tensor dimensions so multiple examples can be stacked into a batch.

```
single image: [3, 224, 224]
batch of 32: [32, 3, 224, 224]
```

Why preprocessing is part of the model

Weights learn from the specific input distribution produced by preprocessing. Changing channel order, image size, crop behavior, or normalization at inference time can reduce accuracy even though the checkpoint has not changed.

Normalization and pretrained weights

Normalization rescales each color channel using a mean and standard deviation. For common ImageNet-pretrained torchvision weights, the familiar values are means [0.485, 0.456, 0.406] and standard deviations [0.229, 0.224, 0.225].

```
normalized_pixel = (pixel - channel_mean) / channel_std
```

Pretrained filters were optimized on images prepared with a compatible input pipeline. Using expected normalization helps maintain a similar input scale. The exact transforms should follow the weight version and the repository's implementation.

Training versus inference transforms

Training may include random augmentation to make the model robust. Validation and inference normally use deterministic transforms so the same image is processed consistently. If the prediction script duplicates transforms separately, those copies should be checked against the training configuration.

Preprocessing is not a cosmetic step. It is part of the trained system's contract and should be versioned alongside the model weights.

Data augmentation

Augmentation applies plausible transformations to training images, such as horizontal flips, small rotations, random crops, or moderate brightness and contrast changes. These transformations create varied training views without requiring a new photograph for every variant.

Purpose

- Reduce memorization of exact training examples.
- Improve robustness to position, scale, and lighting changes.
- Encourage the model to focus on useful breed features rather than incidental background cues.

Risks

- A crop may remove the face, ears, or other important features.
- Excessive rotations or color changes can produce unrealistic images.
- Random augmentation on validation/test data makes metric comparisons noisy.

Augmentation does not add independent biological diversity to the dataset. It changes the appearance of existing images and should reflect realistic variation expected at inference time.

Tensor representation and DataLoader

PyTorch represents images and labels as tensors. A Dataset defines how one indexed example is loaded; a DataLoader groups examples into batches and can shuffle training examples or load them in parallel.

Typical batch

```
images: float tensor [B, 3, 224, 224]
labels: integer tensor [B]
```

For multi-class CrossEntropyLoss, target labels should be integer class indices, usually in the range 0 to 36 for 37 classes.

Why batching helps

- Vectorized operations make better use of the GPU.
- The optimizer updates weights from a batch-level gradient estimate.
- Batch size influences memory usage and optimization behavior.

Common DataLoader problems

A missing file, malformed image, invalid label, or inconsistent tensor shape can cause an exception. Reproducible manifests and early data validation help find these errors before a long training run.

CNN fundamentals

A convolutional neural network learns filters that detect spatial patterns. Early layers often respond to edges and textures; deeper layers combine these into more complex visual features.

Convolution

A kernel slides across a feature map and computes weighted sums. Because the same kernel is reused across spatial locations, convolution can learn useful patterns with fewer parameters than a fully connected layer over every pixel.

ReLU and downsampling

ReLU introduces non-linearity. Pooling or strided convolutions reduce spatial resolution, allowing deeper layers to process more abstract representations with manageable computation.

Why CNNs work for breed recognition

Breed cues may include fur texture, ear shape, face structure, coat markings, and body proportions. CNNs can combine local and global patterns, though they may also exploit shortcuts such as background or image composition.

ResNet-18 and residual learning

ResNet-18 is a convolutional neural network with residual blocks. Instead of learning only a direct transformation, a residual block learns F(x) and adds the original input x when tensor dimensions are compatible.

```
y = F(x) + x
```

The skip connection gives information and gradients a direct path through the block, helping optimization in deeper networks.

Main components

- Initial convolution and batch normalization.
- Residual stages: layer1, layer2, layer3, and layer4.
- Global average pooling.
- Final fully connected classification layer.

The standard ImageNet model ends with 1,000 logits. The pet classifier replaces the final layer with a 37-output layer so that each output corresponds to one pet breed.

Transfer learning

Transfer learning reuses parameters learned on a large source dataset for a related target task. ImageNet-pretrained ResNet-18 has learned general visual representations that can be useful even though the target classes are different.

Feature extraction

Freeze the pretrained backbone and train only a new classifier head. This is computationally cheaper and can work well when the pretrained features already distinguish the target classes.

Fine-tuning

Unfreeze some or all backbone layers and update them on the target dataset. This allows visual representations to adapt to the target domain but increases compute and may overfit if training is not controlled.

Why start with transfer learning?

- Less data and compute are needed than training from random initialization.
- Training can converge faster.
- Pretrained visual features provide a strong baseline.

The Stage 1 repository is centered on the transfer-learning baseline. The later pet-model-deployment repository handles a further fine-tuning stage.

The 37-class classification head

The final fully connected layer maps the feature vector produced by ResNet-18 to one score per target class. For the usual ResNet-18 backbone, the pooled feature vector has 512 values.

```
fc = Linear(512, 37)
```

The output tensor has shape [batch_size, 37]. Each row contains logits for one image. Softmax may be used to display normalized scores; the training loss consumes logits directly.

Why checkpoint shape matters

A standard ImageNet classifier has a 1,000-output head. A 37-class checkpoint cannot be loaded into that head without a matching architecture because the weight dimensions differ.

Index mapping

The predicted integer index must be translated into the exact class name used during training. A mismatch between training and inference mappings is a semantic bug even if model execution succeeds.

Loss: cross-entropy

CrossEntropyLoss is a standard objective for mutually exclusive multi-class classification. It combines a log-softmax operation with negative log likelihood for the correct target class.

Intuitively, the loss is large when the correct breed receives low probability relative to competing classes, and small when the correct class dominates.

```
logits = model(images)
loss = criterion(logits, labels)
loss.backward()
optimizer.step()
```

Why not apply softmax before the loss?

PyTorch's CrossEntropyLoss expects unnormalized logits and applies the numerically stable log-softmax calculation internally. Applying softmax first is unnecessary and changes the expected input.

Loss versus accuracy

Loss captures how strongly the model scores the correct class, while accuracy only checks whether the top prediction is correct. Two models can have the same accuracy but different loss because their confidence distributions differ.

Optimization and gradient descent

Training adjusts model parameters to reduce the loss. Backpropagation calculates gradients of the loss with respect to trainable parameters; the optimizer uses these gradients to update the weights.

Epoch and batch

An epoch is one pass over the training set. A batch is a subset processed in one forward/backward pass. Multiple batches typically make up an epoch.

Learning rate

The learning rate controls the size of optimizer updates. Too high can cause unstable training; too low can lead to slow progress. It is one of the most consequential hyperparameters.

Adam

Adam maintains moving averages of gradients and squared gradients, adapting update scales by parameter. It is widely used for neural-network training. Exact optimizer settings should be read from the training script and recorded with each experiment.

Weight decay can regularize parameters. Optimizer, learning rate, batch size, epochs, and augmentation should be treated as an experimental configuration, not undocumented defaults.

Training loop: line-by-line concepts

A typical PyTorch training epoch follows a stable sequence. The repository's actual implementation should be the final authority for exact function names and arguments.

```
model.train()
for images, labels in train_loader:
    images, labels = images.to(device), labels.to(device)
    optimizer.zero_grad()
    logits = model(images)
    loss = criterion(logits, labels)
    loss.backward()
    optimizer.step()
```

- model.train() enables training behavior for layers such as BatchNorm.
- Moving tensors to the device ensures compatibility with model parameters.
- zero_grad() clears gradients accumulated from the previous batch.
- The forward pass computes class logits.
- The loss compares logits with true labels.
- backward() computes gradients.
- step() updates trainable parameters.

Training metrics should aggregate loss and correct predictions across examples, rather than averaging batch averages blindly when the last batch is smaller.

Validation and best-checkpoint selection

Validation measures the current model without updating its parameters. The model is placed in evaluation mode and gradient recording is disabled. Validation metrics are used to select a checkpoint and tune development decisions.

Why save the best checkpoint?

The last epoch is not guaranteed to be the best. Validation accuracy may peak earlier and decline as the model overfits. Saving the best checkpoint preserves the strongest observed validation result.

model.eval() and no_grad()

model.eval() changes behavior of modules such as BatchNorm and Dropout. torch.no_grad() disables gradient tracking. They are complementary rather than interchangeable.

Avoid test leakage

Checkpoint selection should use validation metrics. The test set should be used only after decisions are complete to provide a less biased final estimate.

Overfitting and generalization

Overfitting occurs when a model learns details specific to training examples that do not generalize to new images. A model can continue improving training loss while validation performance stops improving.

Warning signs

- Training accuracy rises while validation accuracy stalls or falls.
- A small set of familiar backgrounds predicts labels unusually well.
- Errors cluster on new lighting, camera angles, or unusual poses.

Ways to address it

- Use plausible augmentation and weight decay.
- Use pretrained weights and limit trainable parameters when appropriate.
- Select checkpoints using validation performance.
- Inspect errors and consider whether splits contain duplicates or distribution shifts.

Regularization cannot compensate for fundamentally unrepresentative data. A strong test protocol and diverse examples are needed to assess generalization.

Evaluation metrics: accuracy and F1

Accuracy is the proportion of examples classified correctly. It is easy to interpret but can hide poor performance on less common classes.

```
accuracy = correct / total
```

Precision measures how many predictions assigned to a class are correct. Recall measures how many true examples of that class are recovered. F1 is the harmonic mean of precision and recall.

Macro versus weighted averages

- Macro F1 computes F1 per class and averages all classes equally.
- Weighted F1 weights each class by its number of true examples.

Macro F1 is useful for assessing whether minority classes are neglected. Weighted F1 is more influenced by classes with more samples. The repository's README reports the metrics and results associated with its published experiment; do not generalize those results beyond its specific split and checkpoint.

Confusion matrix and error analysis

A confusion matrix counts how often each true class is predicted as each output class. The diagonal represents correct classifications; off-diagonal cells reveal confusions.

Why inspect errors?

Overall accuracy cannot explain whether mistakes are concentrated between visually similar breeds or caused by preprocessing, labels, backgrounds, or poor-quality images.

A useful error-analysis workflow

- Collect misclassified test images and their true/predicted labels.
- Group errors by true class and predicted class.
- Inspect image quality, crop, lighting, background, and visible breed cues.
- Check class mapping and label integrity before changing the architecture.
- Use validation data to test any proposed fix; reserve test results for final reporting.

Error analysis converts a single score into actionable hypotheses about the model and dataset.

Checkpoint files and state dictionaries

A PyTorch state dictionary maps parameter names to learned tensors. Saving it is a common way to store model weights. To load it correctly, instantiate the same architecture and classifier shape first.

What weights alone do not capture

- The full preprocessing pipeline.
- The class-index-to-name mapping.
- The dataset split and evaluation protocol.
- All package versions and training hyperparameters.

For reproducible handoff, save metadata alongside the checkpoint: architecture, number of classes, class mapping, transforms, metrics, and code revision. The checkpoint file name referenced in the README should match the actual path expected by the scripts.

Single-image prediction

Prediction applies the trained model to an image that was not part of the optimization process. The script loads the checkpoint, creates the model, applies inference transforms, performs a forward pass, and maps the top class index to a breed name.

Inference sequence

- Open the image and convert it to RGB.
- Resize/crop and normalize using the expected transforms.
- Add a batch dimension with unsqueeze(0).
- Move input tensor and model to the same device.
- Run model.eval() under torch.no_grad().
- Rank class scores and map indices to breed names.

A displayed confidence score is typically a softmax score. It represents relative model output, not a calibrated guarantee that the prediction is correct.

CPU, CUDA, and memory

PyTorch can run on CPU or a CUDA-capable NVIDIA GPU. GPU acceleration is useful for convolution-heavy training, while CPU inference can be convenient for simple deployment.

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model.to(device)
images = images.to(device)
```

Model parameters and input tensors must be on the same device. A CPU/CUDA mismatch causes a runtime error.

Memory constraints

VRAM use depends on model parameters, batch size, input dimensions, intermediate activations, and other GPU workloads. If memory runs out, reduce batch size or use other appropriate memory strategies.

The laptop GPU is a training resource, not a requirement for understanding the pipeline. Device selection should remain explicit so the same scripts can run on a CPU-only machine.

Source-code responsibilities and design

The repository separates the responsibilities of dataset loading, model construction, training, evaluation, and prediction. The exact file names in the repository should be checked before adding documentation or changing the code.

Dataset module

Reads manifests, decodes images, applies transforms, and returns image/label pairs.

Model module

Constructs the ResNet-18 backbone and replaces its output layer for 37 classes.

Training module

Creates data loaders, computes loss and gradients, validates epochs, and saves a selected checkpoint.

Evaluation module

Loads a fixed checkpoint and computes held-out metrics without changing weights.

Prediction module

Runs the compatible preprocessing and model forward pass for a new image.

This separation makes individual pieces easier to test and helps prevent training-time logic from diverging from inference-time logic.

Common failures and debugging

State-dictionary size mismatch

Likely cause: loading a 37-class checkpoint into the original 1,000-class head. Replace the classifier before loading the checkpoint.

Wrong breed names

Likely cause: inconsistent class-index mapping or one-based/zero-based labels. Compare the training manifest and prediction mapping.

CUDA device mismatch

Likely cause: model and input tensors reside on different devices. Move both to the selected device.

Poor results on new photos

Check RGB conversion, image transforms, checkpoint selection, class mapping, and distribution shift before assuming the architecture needs replacement.

DataLoader crashes

Check file existence, image decoding, target label range, tensor shape, and dependency versions.

Debug from the data inward: file → decoded image → transform → label → model/checkpoint → logits → class mapping → metric.

Limitations and next improvements

The pipeline demonstrates core supervised image classification and transfer learning. Its reported metrics are tied to the documented dataset split, model checkpoint, and evaluation procedure; they do not prove performance on every real-world image.

Model-side improvements

- Report per-class precision/recall and a confusion matrix.
- Compare frozen-feature extraction against controlled fine-tuning.
- Track experiments, hyperparameters, and checkpoint metadata.
- Measure prediction latency and confidence calibration.

Data and engineering improvements

- Validate manifest integrity and detect duplicate images.
- Add automated tests for label mapping and inference preprocessing.
- Use continuous integration for linting and test execution.
- Document dependency versions and a clean reproducible setup.

Keep future improvements separate from the existing implementation in project documentation. A proposed feature should not be described as completed until it is implemented and tested.

Viva questions: core concepts

1. What problem does the project solve?

It predicts one of 37 cat/dog breed labels from an input image.

2. Why use transfer learning?

Pretrained visual features provide a strong starting point when the target dataset is relatively small.

3. Why ResNet-18?

It offers a practical balance of model capacity, compute cost, and availability of pretrained weights.

4. What is a logit?

An unnormalized score produced for a class before softmax normalization.

5. Why CrossEntropyLoss?

It is suitable for mutually exclusive multi-class classification and expects logits plus integer class labels.

6. What is an epoch?

One full pass through the training dataset.

7. Why use augmentation?

To expose the model to plausible variations and reduce memorization of exact training images.

Viva questions: evaluation and inference

8. Why separate validation and test sets?

Validation supports model selection; test provides a final estimate after development decisions are complete.

9. What does macro F1 tell us?

It averages class F1 scores equally and highlights performance across less common classes.

10. What is a checkpoint?

A saved model parameter state, often accompanied by metadata needed to reconstruct the model.

11. Why use model.eval()?

It sets inference behavior for modules such as BatchNorm and Dropout.

12. Why use torch.no_grad()?

It disables autograd tracking during inference, reducing memory and computation.

13. Why must class mapping be stable?

The predicted integer index must map to the same breed name used when the model was trained.

14. Does a softmax score guarantee correctness?

No. The model can be overconfident, especially on unfamiliar inputs.

15. What would you improve next?

I would strengthen class-level error analysis, reproducibility, inference tests, and experiment tracking.

Quick reference and study plan

Core concepts to be able to explain

- Dataset manifests and split leakage.
- RGB conversion, resizing, normalization, and augmentation.
- ResNet residual connections and the 37-class head.
- Transfer learning versus fine-tuning.
- Cross-entropy, gradients, optimizers, epochs, and learning rates.
- Validation-based checkpoint selection and held-out testing.
- Accuracy, precision, recall, macro F1, and confusion matrices.
- State-dictionary loading and inference preprocessing.

Repository

https://github.com/ApexHyper2007/pet-breed-ml-pipeline

Read the source in this order: dataset loading → model definition → training loop → validation/checkpoint saving → test evaluation → prediction. Follow each tensor from the decoded image to the final breed name.

Best viva preparation: for each design choice, explain what it does, why it is used, what can go wrong, and how you would verify it.